INTERNOVA

University Internship & Industry Matching Portal

Development Documentation

Frontend (Vite + React) · Backend (ASP.NET Core Web API)

Project	Internova — Internship Matching Portal
Tech Stack	Vite + React | ASP.NET Core Web API
Architecture	Clean Architecture (Core / Infrastructure / API)
Database	MySQL 8+ via Entity Framework Core
Cloud	Azure Blob Storage (Resume uploads)
Branch	develop
Date	2026

Confidential | Internal Use Only

Table of Contents

1. Code-Level Documentation 4
   1.1 Backend (.NET) — XML Summary Comments 4
      Example — AuthController XML Docstring 4
      Example — JWT Startup Guard (Program.cs) 4
   1.2 Frontend (React) — JSDoc & Component Comments 4
      Example — ProtectedRoute JSDoc 4
      Example — LoginPage Redirect Rationale 5
   1.3 Self-Documenting Code Conventions 5
2. Architecture & Implementation Details 6
   2.1 High-Level System Overview 6
   2.2 Backend — Clean Architecture Layering 6
   2.3 Backend — Key Modules 6
   2.4 Frontend — Routing & Role Gating 7
   2.5 Dependency List & Rationale 7
      Backend — NuGet Packages 7
      Frontend — npm Packages 7
3. API / Interface Contracts 8
   3.1 Health Endpoints 8
   3.2 Authentication Endpoints 8
      POST /api/auth/register — Request Schema 8
      POST /api/auth/login — Response Schema (200 OK) 8
   3.3 Student Profile & Resume Upload 8
      StudentProfile Entity — Schema 9
4. Alignment with Acceptance Criteria 10
   4.1 Acceptance Criteria Mapping 10
   4.2 Technical Decision Justifications 10
      Why JWT over Server-Side Sessions? 10
      Why Azure Blob Storage over Local File Storage? 10
      Why Idempotent DatabaseInitializer over EF Core Migrations? 10
5. Implementation Readiness 11
   5.1 Backend — Build & Run 11
      Step 1 — Configure appsettings.Development.json 11
      Step 2 — Configure JWT Secrets 11
      Step 3 — Build & Run 11
      Step 4 — Verify 11
   5.2 Frontend — Build & Run 11
   5.3 Testing Hooks & QA Notes 12
      Testability Design 12

Manual QA Scenarios.....12

1. Code-Level Documentation

This section details the commenting conventions, docstring standards, and self-documenting coding practices adopted across both the backend (.NET) and frontend (React) codebases to ensure long-term maintainability and developer onboarding efficiency.

1.1 Backend (.NET) — XML Summary Comments

All controllers, services, and domain models include XML-style summary comments that describe their purpose, parameters, and return values. These comments are consumed by Swagger/OpenAPI to generate interactive API documentation automatically.

> *Design Decision: XML doc comments were chosen over plain inline comments for the backend because they integrate directly with Swashbuckle to produce live API documentation — removing the need to maintain a separate API reference document.*

Example — AuthController XML Docstring

```
/// <summary>
/// Registers a new user in the system.
/// Validates the request, checks for duplicate emails,
/// and enforces role creation rules (Admin self-registration blocked).
/// </summary>
/// <param name="request">RegisterRequest containing fullName, email, password,
role.</param>
/// <returns>201 Created with userId on success; 400/409 on failure.</returns>
[HttpPost("register")]
public async Task<IActionResult> Register([FromBody] RegisterRequest request) { ... }
```

Example — JWT Startup Guard (Program.cs)

```
// FAIL-FAST: If JWT secret is missing at startup, throw immediately.
// This prevents silent auth failures in production where tokens
// would be signed with an empty key and accepted by any issuer.
var jwtKey = builder.Configuration["Jwt:Key"]
    ?? throw new InvalidOperationException(
        "Jwt:Key is not configured. Set it via user-secrets or env var.");
```

1.2 Frontend (React) — JSDoc & Component Comments

Route-level components, the authentication context, and core utilities include JSDoc blocks that describe their behaviour, props, and return values. Auth-specific files include rationale comments explaining the timing of React state updates and redirect logic.

Example — ProtectedRoute JSDoc

```
/**
 * ProtectedRoute — wraps a route to enforce authentication and role-based access.
 *
 * @param {React.ReactNode} children — The protected page component to render.
 * @param {string[]} allowedRoles — Roles permitted to view this route.
 *
 * Behaviour:
 * - Unauthenticated users are redirected to /login.
 * - Authenticated users with the wrong role are redirected to their dashboard.
```

```
 * - Authorised users are rendered normally.
 */
const ProtectedRoute = ({ children, allowedRoles }) => { ... };
```

Example — LoginPage Redirect Rationale

```
// We call setUser() first and navigate() inside a useEffect that watches
// the user state. Calling navigate() immediately after setUser() inside the
// same handler risks reading stale state before React flushes the update,
// which would redirect to the wrong dashboard for some role transitions.
useEffect(() => {
    if (user) navigate(getDashboardPath(user.role));
}, [user]);
```

1.3 Self-Documenting Code Conventions

Both repositories follow a consistent naming convention that reduces the need for explanatory comments on straightforward logic.

- **User, StudentProfile, BlobStorageService** Entity names are domain-aligned:
- **api/auth, api/health, api/student** Route prefixes are explicit:
- **isAuthenticated, isLoading** Boolean flags are phrased as questions:
- **UploadResumeAsync(), GetUserByEmailAsync()** Async functions are suffixed:

2. Architecture & Implementation Details

Internova comprises two separate repositories: a React SPA frontend and an ASP.NET Core Web API backend. The separation of concerns allows each layer to be developed, tested, and deployed independently.

2.1 High-Level System Overview

Frontend	Backend
Vite + React + TailwindCSS	ASP.NET Core Web API (.NET 10)
Role-based portals (Student / Company / Admin)	Clean Architecture: Core / Infrastructure / API
JWT-authenticated routes via ProtectedRoute	MySQL 8 via EF Core (Pomelo provider)
Axios interceptor attaches Bearer token	Azure Blob Storage for resume PDF uploads

2.2 Backend — Clean Architecture Layering

> *Why Clean Architecture? Isolating domain logic in Core means it can be unit-tested without spinning up a database or HTTP server. Infrastructure implementations can be swapped (e.g. replacing MySQL with PostgreSQL) without touching controller code.*

Layer	Project	Responsibilities
Domain	Internova.Core	Entities (User, StudentProfile) and repository interfaces. No external NuGet dependencies.
Infrastructure	Internova.Infrastructure	EF Core AppDbContext, MySQL connectivity, DatabaseInitializer, and BlobStorageService. Depends only on Core.
Presentation	Internova.Api	ASP.NET controllers, JWT middleware, Swagger, CORS policy, and the DI composition root (Program.cs).

2.3 Backend — Key Modules

Module	Responsibility
Program.cs	DI setup, JWT auth configuration, Swagger, CORS policy, startup DB initialization. Fail-fast JWT key guard is located here.
DatabaseInitializer	Idempotently creates internova_db and tables (Users, StudentProfiles) at startup — no migration tooling required.
AppDbContext	EF Core DbContext mapping entities to MySQL tables. Never referenced directly by controllers (dependency inversion).
AuthController	Register/login/me endpoints. GenerateJwt() embeds user_id, email, and role claims. Admin self-registration blocked.
BlobStorageService	Validates (PDF + max 5 MB) and uploads resume files to private Azure Blob container. Returns Blob URL.

2.4 Frontend — Routing & Role Gating

Client-side routing is managed by React Router v6. Routes are divided into two categories:

- Public routes — /landing, /login, /register. Accessible without a JWT.
- Protected routes — all dashboard and profile pages. Wrapped in ProtectedRoute, which verifies the JWT and role before rendering.

> *ProtectedRoute enforces two guards: unauthenticated users are sent to /login; role-mismatched users are redirected to their correct dashboard. This prevents horizontal privilege escalation at the UI layer.*

2.5 Dependency List & Rationale

Backend — NuGet Packages

Package	Used For	Rationale
Pomelo.EntityFrameworkCore.MySql	EF Core MySQL provider	Best-maintained open-source EF Core provider for MySQL 8. Active community support.
MySqlConnector	Low-level MySQL connectivity	Used by health checks and DatabaseInitializer for raw SQL outside EF Core.
Azure.Storage.Blobs	Azure Blob Storage SDK	First-party Microsoft SDK. Provides async streaming and SAS URL generation.
BCrypt.Net-Next	Password hashing	Intentionally slow and salted — computationally expensive for dictionary attacks.
Swashbuckle.AspNetCore	Swagger / OpenAPI	Auto-generates interactive API docs from XML comments. Reduces manual doc burden.
Microsoft.AspNetCore.Authentication.JwtBearer	JWT auth middleware	Validates Bearer tokens on protected routes; extracts claims into HttpContext.User.

Frontend — npm Packages

Package	Used For	Rationale
React + React Router v6	SPA rendering & routing	Industry standard. Router v6 data APIs simplify nested layouts and role-based redirects.
Axios	HTTP client	Request interceptors centrally attach the Authorization header — no boilerplate repetition.
jwt-decode	JWT claim parsing	Lightweight library to decode JWT payload client-side: user_id, email, role for AuthContext.
TailwindCSS	Utility-first styling	Eliminates CSS file context-switching. Consistent design system across all role portals.
Formik	Form state management	Reduces boilerplate for controlled inputs, validation, and submission across forms.

3. API / Interface Contracts

All backend routes are prefixed with /api. The Vite development server is configured to proxy /api requests to the ASP.NET backend, eliminating CORS issues during local development.

3.1 Health Endpoints

Method	Endpoint	Auth	Description
GET	/api/health/ping	None	Liveness check. Returns 200 OK with { "status": "ok" }.
GET	/api/health/db	None	Readiness check. Verifies MySQL connectivity and presence of the Users table. Returns 200 OK or 503.

3.2 Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	Register a new user. Returns 201 + userId; 400 on validation failure; 409 on duplicate email.
POST	/api/auth/login	None	Authenticate a user. Returns 200 with AuthResponse (token, userId, email, role) or 401.
GET	/api/auth/me	Bearer JWT	Returns list of claims from the authenticated user's JWT.

POST /api/auth/register — Request Schema

```
{
  "fullName": "Janeesha Perera",
  "email":    "janeesha@university.ac.lk",
  "password": "••••••••",
  "role":     "Student"   // "Student" | "Company"  —  Admin self-registration blocked
}
```

POST /api/auth/login — Response Schema (200 OK)

```
{
  "token":  "<signed-jwt>",
  "userId": 1,
  "email":  "janeesha@university.ac.lk",
  "role":   "Student"
}

// JWT Payload (decoded):
// { user_id: 1, email: "...", role: "Student", exp: <unix-ts> }
```

3.3 Student Profile & Resume Upload

Method	Endpoint	Auth	Description
PUT	/api/student/profile	Bearer JWT	Update student profile and/or upload resume. Accepts multipart/form-data. Returns 200/204 with updated ResumeUrl.

StudentProfile Entity — Schema

Field	Type	Description
Id	int	Primary key
UserId	int	FK → Users.Id
UniversityId	string	University identifier
Department	string	Department / faculty
GPA	decimal	Academic GPA
Skills	string	Comma-separated skill tags
ResumeUrl	string	Azure Blob URL of uploaded PDF
CreatedAt	datetime	Record creation timestamp
UpdatedAt	datetime	Last update timestamp

Resume Validation Rules: PDF content-type AND .pdf extension must both be present. Maximum file size: 5 MB. Filenames are made unique using studentId + UTC timestamp to prevent overwrites in the blob container.

4. Alignment with Acceptance Criteria

This section maps implemented modules to Sprint Backlog acceptance criteria and documents the technical decisions made to satisfy the business value defined for each story.

4.1 Acceptance Criteria Mapping

Story ID	Module / Component	Implementation Detail
CSP-10	AuthController.GenerateJwt()	JWT embeds user_id, email, and role claims. Satisfies the requirement that the API returns a role-bearing token for downstream authorisation.
CSP-10	ProtectedRoute (Frontend)	Unauthenticated users redirected to /login. Role-mismatched users redirected to their correct dashboard — prevents horizontal privilege escalation.
CSP-10	LoginPage useEffect redirect	navigate() is called inside a useEffect watching user state to guarantee React has flushed the state update before reading the role.
CSP-15	StudentProfile entity	Defines the profile schema including ResumeUrl, aligning with the BA requirement that student profiles store a verifiable resume link.
CSP-15	DatabaseInitializer	Creates StudentProfiles table idempotently at startup — no manual migration steps required.
CSP-15	BlobStorageService.UploadResumeAsync()	Validates and stores resume PDFs in a private Azure Blob container, returning a Blob URL to persist in StudentProfiles.ResumeUrl.
CSP-15	PUT /api/student/profile	Multipart endpoint integrates profile text fields with optional resume upload in a single request, matching the sprint task definition.

4.2 Technical Decision Justifications

Why JWT over Server-Side Sessions?

The BA's business value for CSP-10 required role-based access across a decoupled frontend and backend. JWTs are stateless: the backend does not need to maintain a session store, reducing infrastructure cost and enabling horizontal scaling. The role claim embedded in the token allows the frontend ProtectedRoute to make access decisions without an additional round-trip to the server.

Why Azure Blob Storage over Local File Storage?

Resume files uploaded in development environments are frequently lost when containers or servers are restarted. Azure Blob Storage provides durable, environment-independent file persistence. Using a private container (rather than public) satisfies the implied confidentiality requirement for student CV data — files are not publicly accessible by URL without an authorised SAS token.

Why Idempotent DatabaseInitializer over EF Core Migrations?

EF Core migration files accumulate over time and require careful ordering when applied to a new environment. For a sprint-cadence project, the DatabaseInitializer approach bootstraps a runnable environment from zero with a single dotnet run — removing a common onboarding blocker for new team members and QA engineers.

5. Implementation Readiness

5.1 Backend — Build & Run

Prerequisites: .NET 10 SDK, MySQL 8+, an Azure Storage account (for resume upload testing).

Step 1 — Configure appsettings.Development.json

Create the file (git-ignored) in the Internova.Api project root:

```
{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost;Port=3306;Database=internova_db;User=root;Password=YOUR_PASSWORD;"
  },
  "AzureBlobStorage": {
    "ConnectionString": "DefaultEndpointsProtocol=https;AccountName=...;"
  }
}
```

Step 2 — Configure JWT Secrets

Use .NET user-secrets to avoid committing sensitive values to source control:

```
dotnet user-secrets init
dotnet user-secrets set "Jwt:Key"      "<strong-random-secret-min-32-chars>"
dotnet user-secrets set "Jwt:Issuer"   "Internova"
dotnet user-secrets set "Jwt:Audience" "InternovaUsers"
```

Step 3 — Build & Run

```
dotnet build
dotnet run --project Internova.Api
```

Step 4 — Verify

```
# Swagger UI
http://localhost:<port>/swagger

# Liveness check
GET http://localhost:<port>/api/health/ping
# Expected: { "status": "ok" }

# Database readiness check
GET http://localhost:<port>/api/health/db
# Expected: 200 OK (tables created automatically on first run)
```

5.2 Frontend — Build & Run

Prerequisites: Node.js 18+, npm 9+.

```
# 1. Copy environment template
cp .env.example .env.local

# 2. Set the backend URL in .env.local
# VITE_API_BASE_URL=http://localhost:<backend-port>
# (Leave blank to use the Vite proxy defined in vite.config.js)

# 3. Install dependencies
npm install

# 4. Start development server
npm run dev
# Default: http://localhost:5173
```

5.3 Testing Hooks & QA Notes

Testability Design

All external dependencies (database, blob storage) are accessed through interface abstractions (IBlobStorageService, IUserRepository). This enables unit tests to inject mock implementations without requiring a live database or Azure connection. Controllers accept injected interfaces via constructor DI, making them independently testable.

Manual QA Scenarios

Scenario	Action	Expected Result
Register Student	POST /api/auth/register with role Student	201 Created + userId
Register Admin blocked	POST /api/auth/register with role Admin	400 Bad Request
Login + JWT	POST /api/auth/login, decode JWT	user_id, email, role claims present
Role redirect	Login as Student, navigate to /company/dashboard	Redirected to /student/dashboard
Valid resume upload	PUT /api/student/profile with PDF < 5 MB	200/204 + ResumeUrl saved in DB
Invalid file type	Upload a .docx file	400 Bad Request with validation error
Oversized file	Upload PDF > 5 MB	400 Bad Request with size error
DB health check	Stop MySQL, GET /api/health/db	503 Service Unavailable

> *Swagger UI at /swagger provides immediate interactive testing for all auth and health endpoints. XML doc comments on controllers are surfaced directly in the UI — giving QA engineers endpoint-level documentation without a separate reference document.*